

实战：LangChain 版 Auto-GPT

彭靖田

Google Developer Experts
LangChain Developer



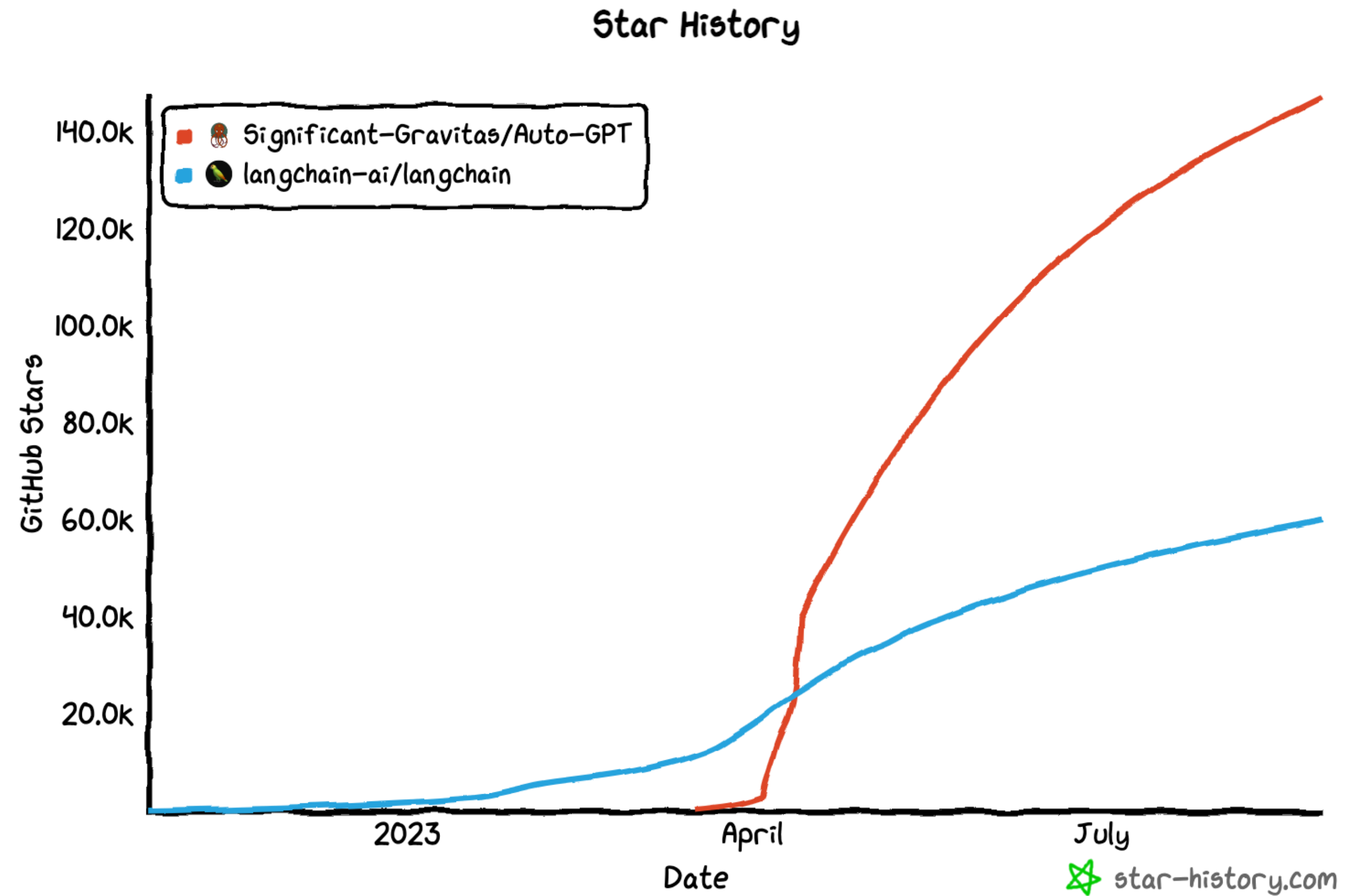
目录

- Auto-GPT 项目定位与价值解读
 - Auto-GPT 开源项目介绍
 - Auto-GPT 定位：一个自主的 GPT-4 实验
 - Auto-GPT 价值：一种基于 Agent 的 AGI 尝试
- LangChain 版 Auto-GPT 技术方案与架构设计
 - 深入理解 LangChain Agents
 - LangChain Experimental 模块
 - Auto-GPT 自主智能体设计
 - Auto-GPT Prompt 设计
 - Auto-GPT Memory 设计
 - 深入理解 LangChain VectorStore
 - Auto-GPT OutputParser 设计
- 实战 LangChain 版 Auto-GPT

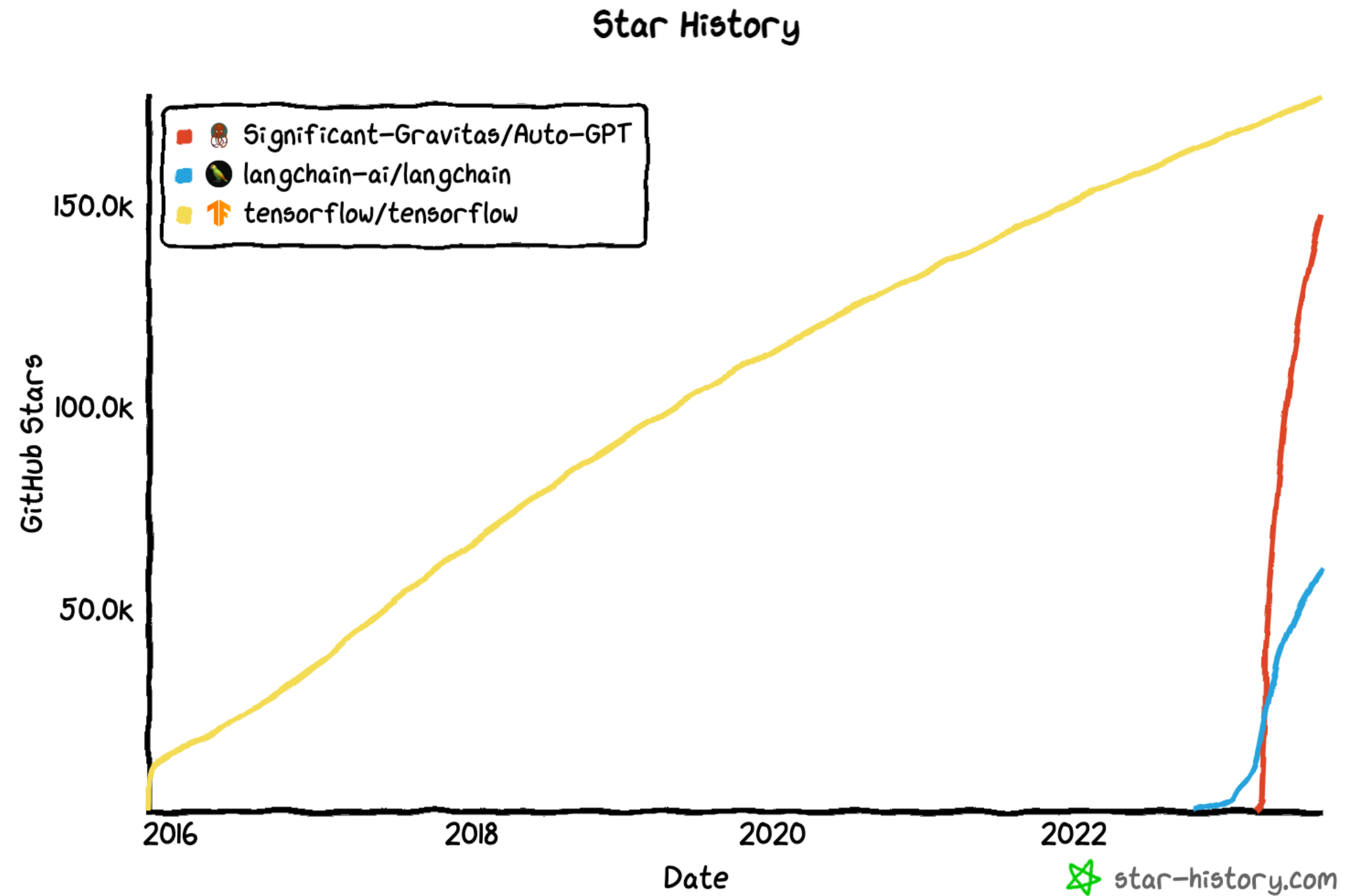
实战 LangChain 版 Auto-GPT

Auto-GPT 项目定位与价值解读

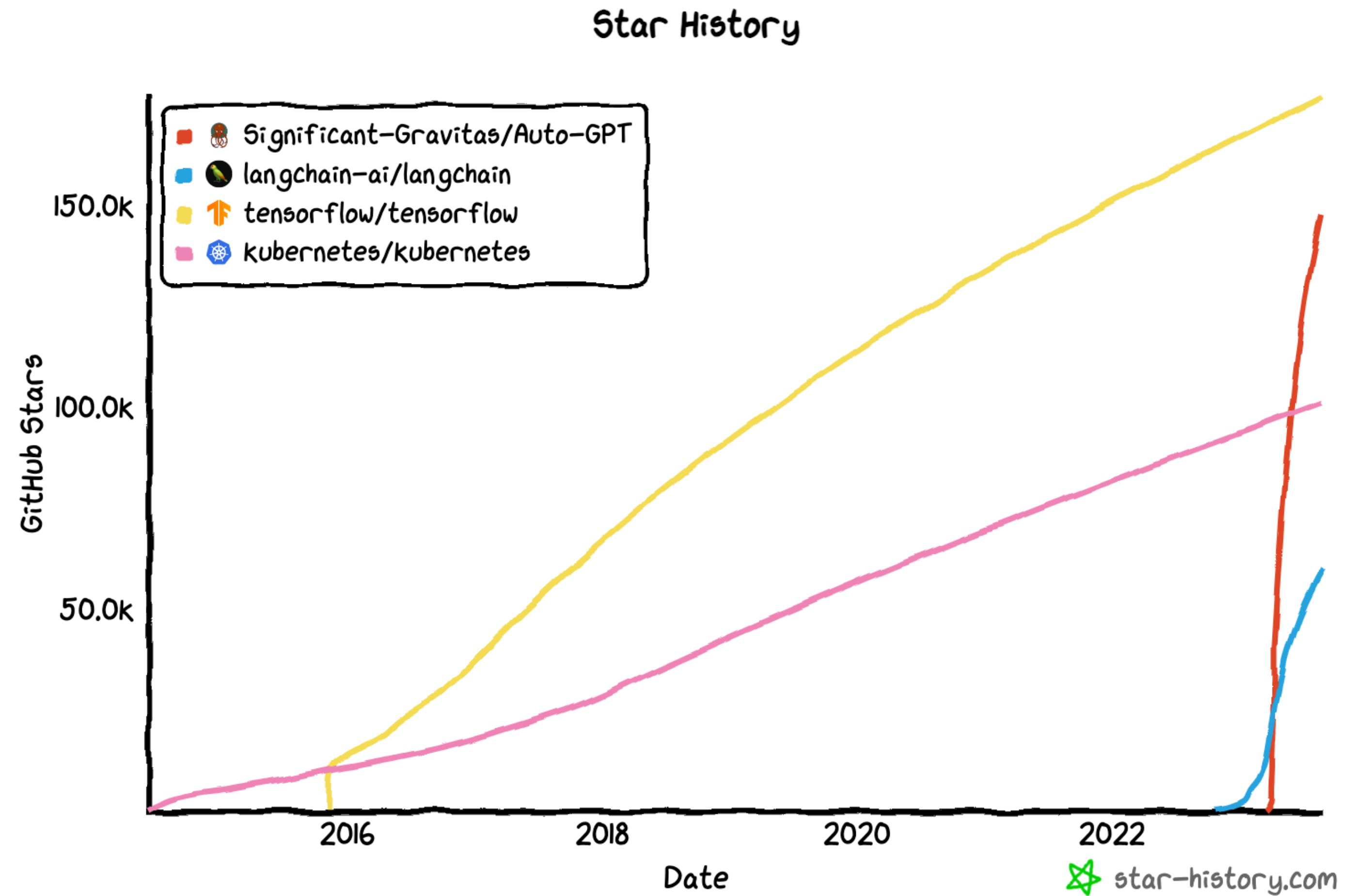
Auto-GPT 开源项目



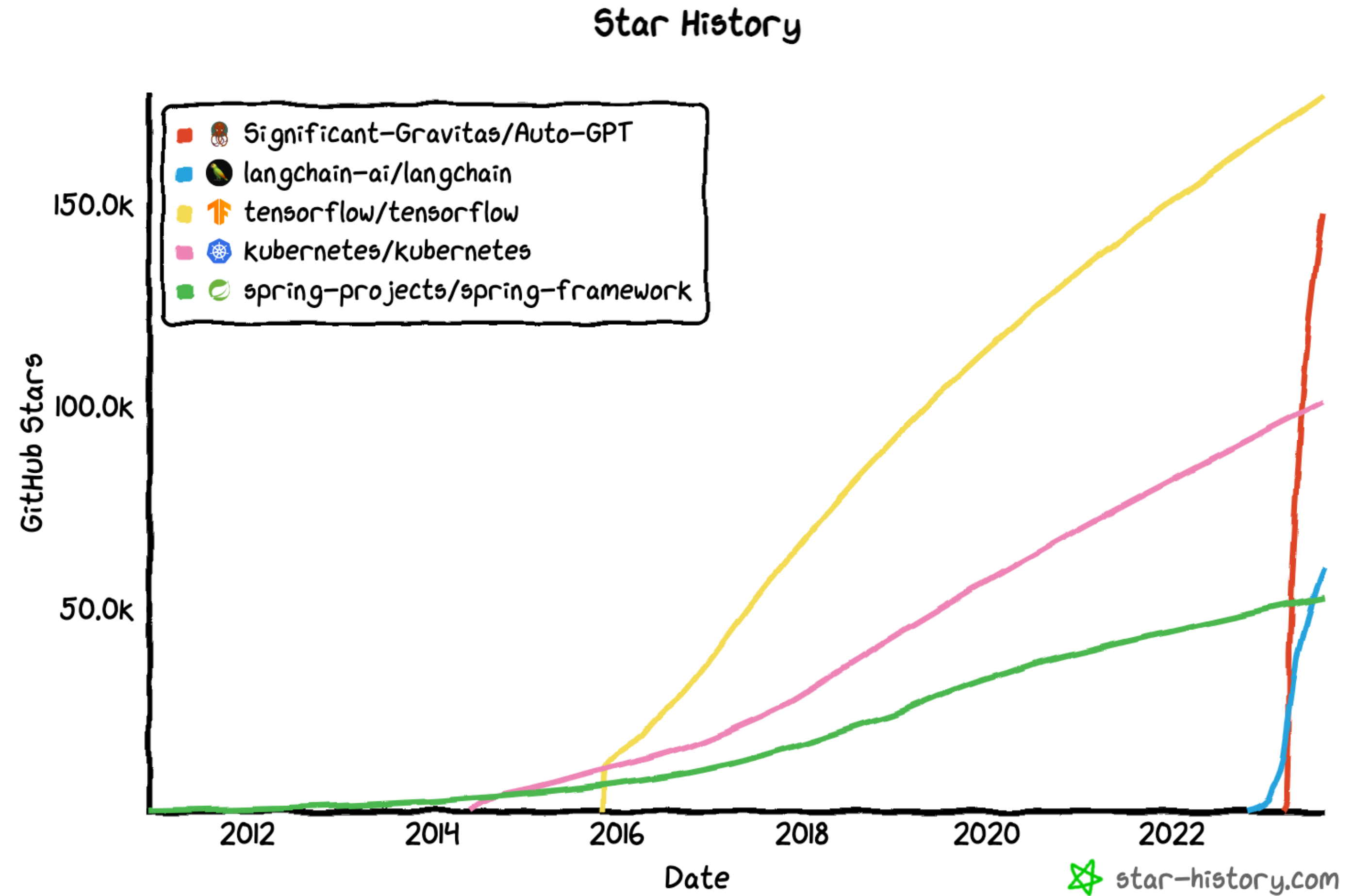
LLM vs DL



AI vs K8s

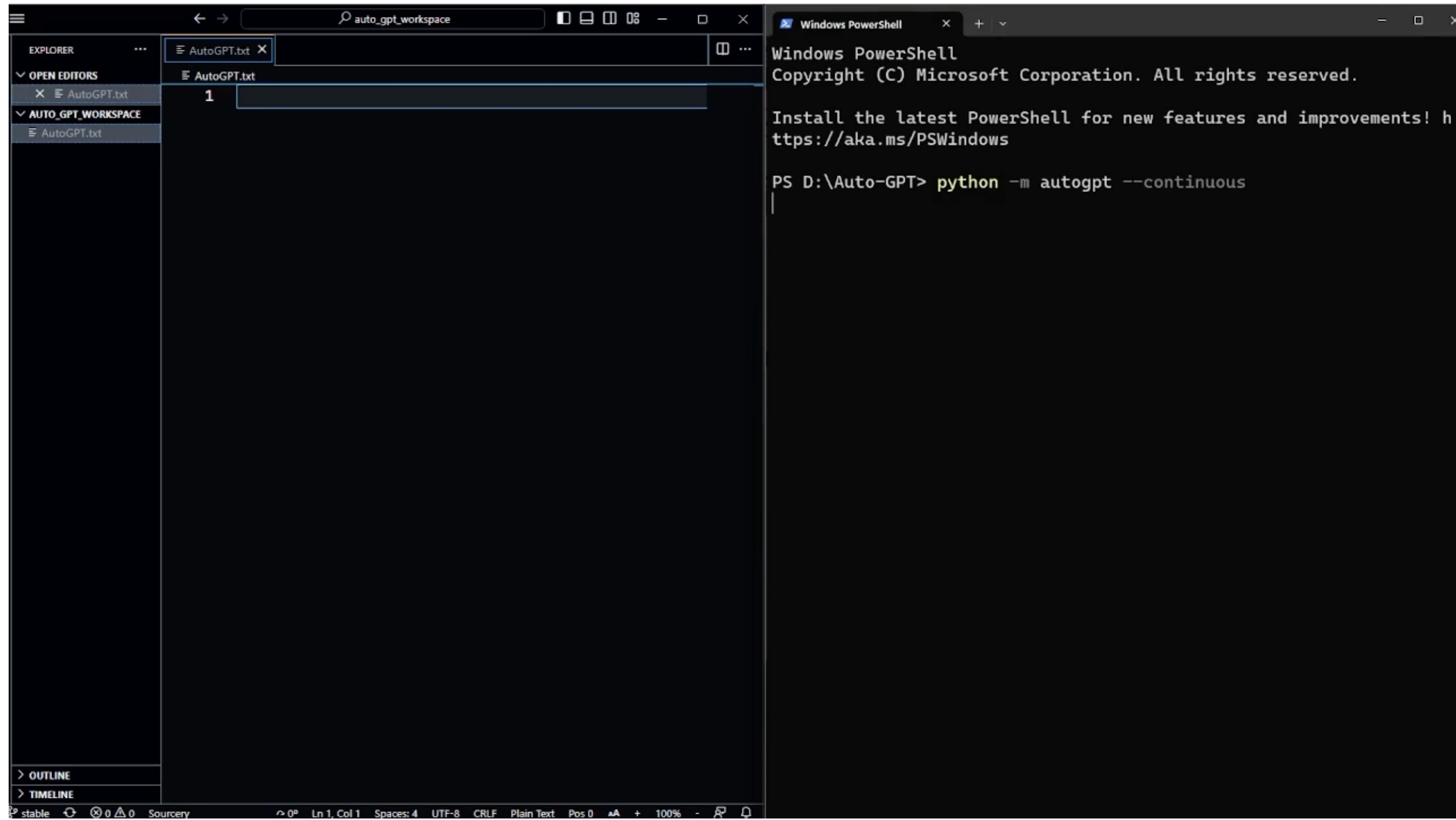


开源项目 见证技术变迁



Auto-GPT 定位：一个自主的 GPT-4 实验

- Auto-GPT是一个实验性的开源应用程序，展示了GPT-4语言模型的能力。
- Auto-GPT由GPT-4驱动，通过链式（Chains）连接LLM“思维（thoughts）”来自主地实现您设定的任何目标。



The screenshot displays a Visual Studio Code editor window with the 'auto_gpt_workspace' open. The Explorer sidebar on the left shows the file structure, including 'AutoGPT.txt' under the 'AUTO_GPT_WORKSPACE' folder. The main editor area shows the 'AutoGPT.txt' file with a cursor at line 1. To the right, a Windows PowerShell terminal window is open, showing the command 'python -m autogpt --continuous' being executed in the directory 'D:\Auto-GPT'.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS D:\Auto-GPT> python -m autogpt --continuous
```


Auto-GPT 定位：一个自主的 GPT-4 实验

- Auto-GPT是一个实验性的开源应用程序，展示了GPT-4语言模型的能力。
- Auto-GPT由GPT-4驱动，通过链式（Chains）连接LLM“思维（thoughts）”来自主地实现您设定的任何目标。

Features

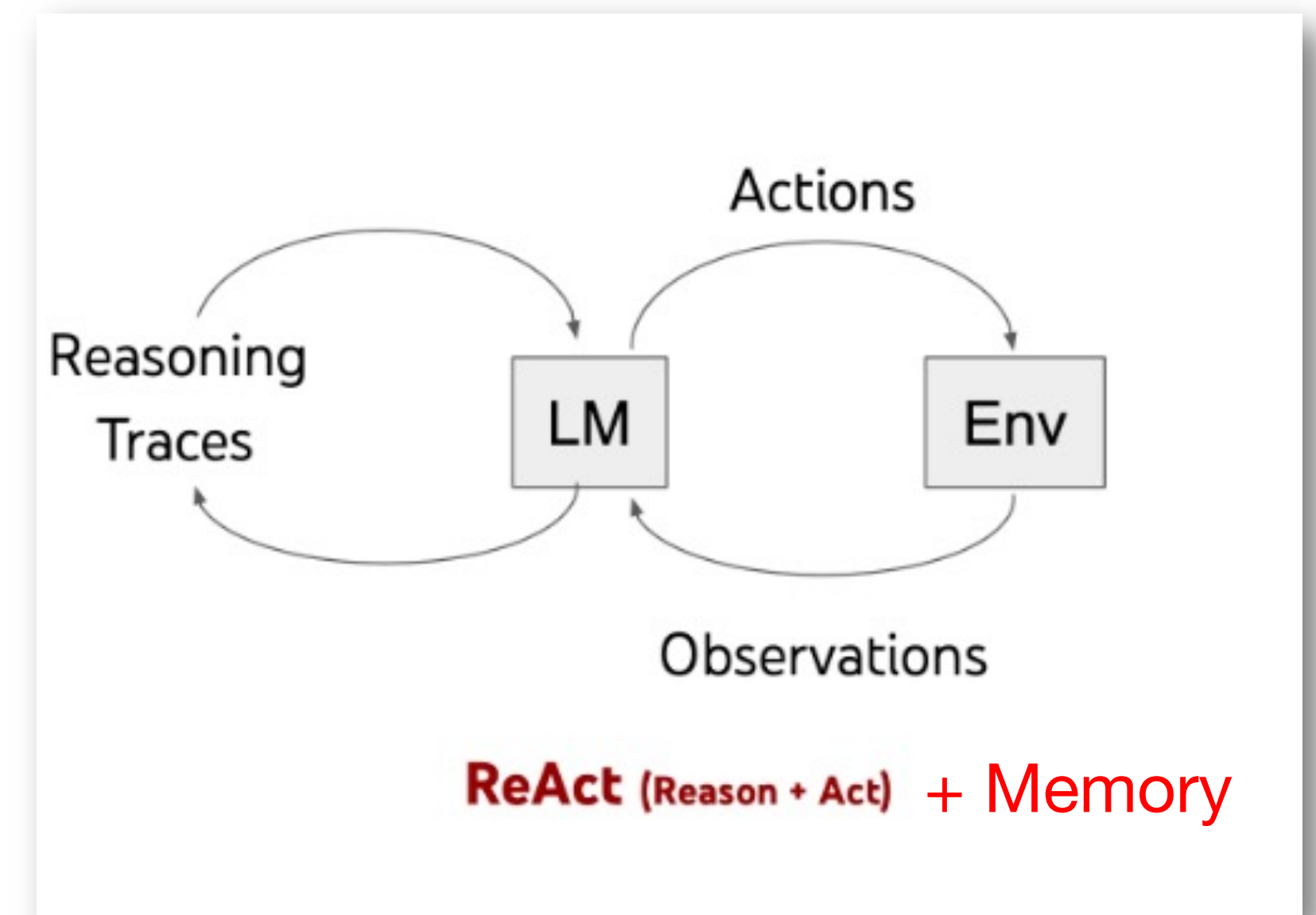
-  Internet access for searches and information gathering
-  Long-term and short-term memory management
-  GPT-4 instances for text generation
-  Access to popular websites and platforms
-  File storage and summarization with GPT-3.5
-  Extensibility with Plugins

Auto-GPT 价值：一种基于 Agent 的 AGI 尝试

- Auto-GPT是一个实验性的开源应用程序，展示了GPT-4语言模型的能力。
- Auto-GPT由GPT-4驱动，通过链式（Chains）连接LLM“思维（thoughts）”来自主地实现您设定的任何目标。
- 作为GPT-4完全自主运行的首批示例之一，Auto-GPT 推动了人工智能所能达到的边界。

🚀 Features

- 🌐 Internet access for searches and information gathering
- 💾 Long-term and short-term memory management
- 🧠 GPT-4 instances for text generation
- 🔗 Access to popular websites and platforms
- 📁 File storage and summarization with GPT-3.5
- 🔌 Extensibility with Plugins

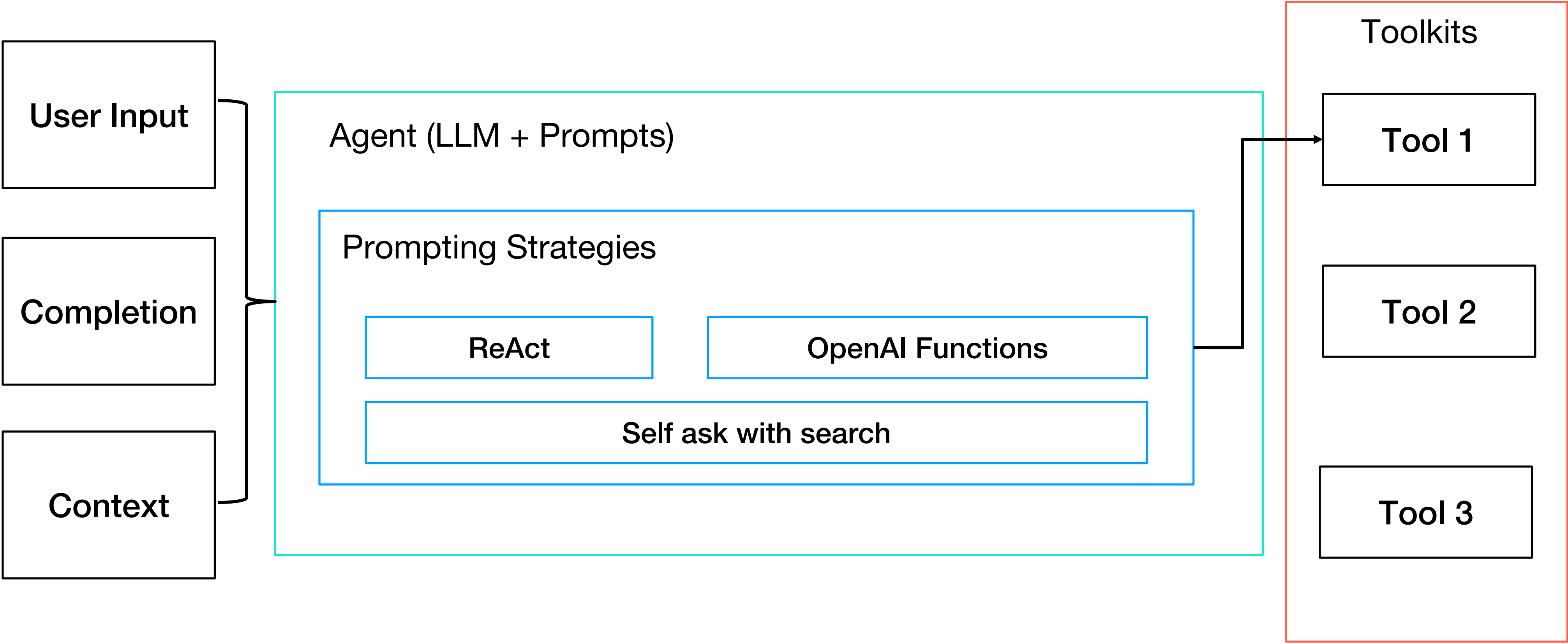
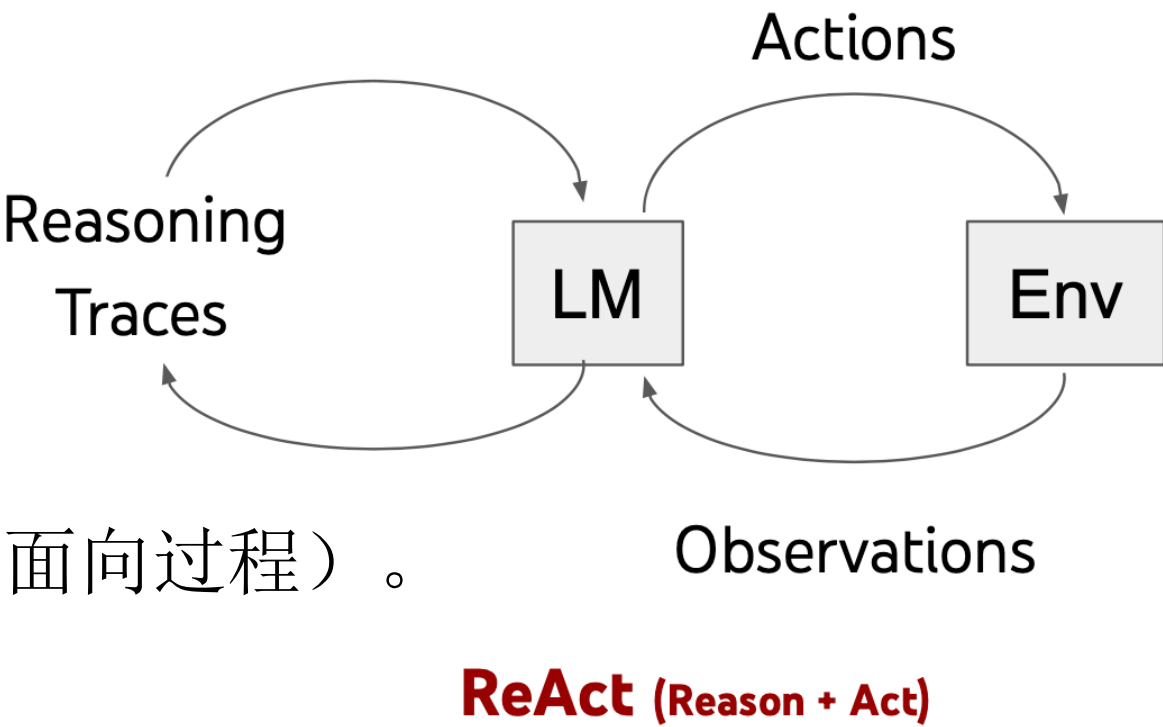


LangChain 版 Auto-GPT 技术方案与架构设计

温故：LangChain Agents 模块设计

代理的核心思想是使用LLM来选择一系列要执行的动作。

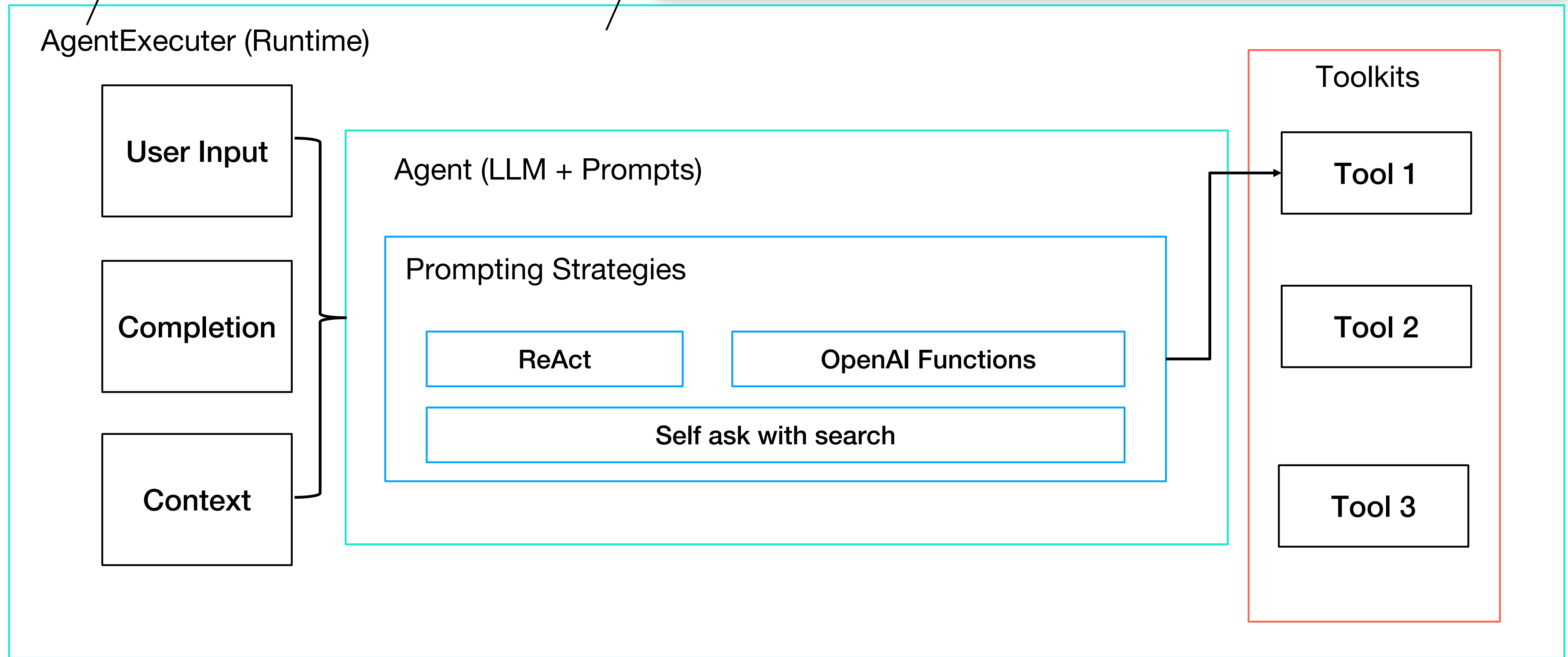
- 在链式结构（Chains）中，一系列动作执行是硬编码的（ SequentialChain 和 RouterChain 也仅实现了面向过程）。
- 在代理（Agents）中，语言模型被用作推理引擎，以确定应该采取哪些动作以及执行顺序。



温故：LangChain Agents 模块设计

- Plan-and-execute Agent
- BabyAGI
- AutoGPT

```
next_action = agent.get_action(...)
while next_action != AgentFinish:
    observation = run(next_action)
    next_action = agent.get_action(..., next_action, observation)
return next_action
```



LangChain Agents 生态

基于LLM的智能代理是通往 AGI 的有效探索。

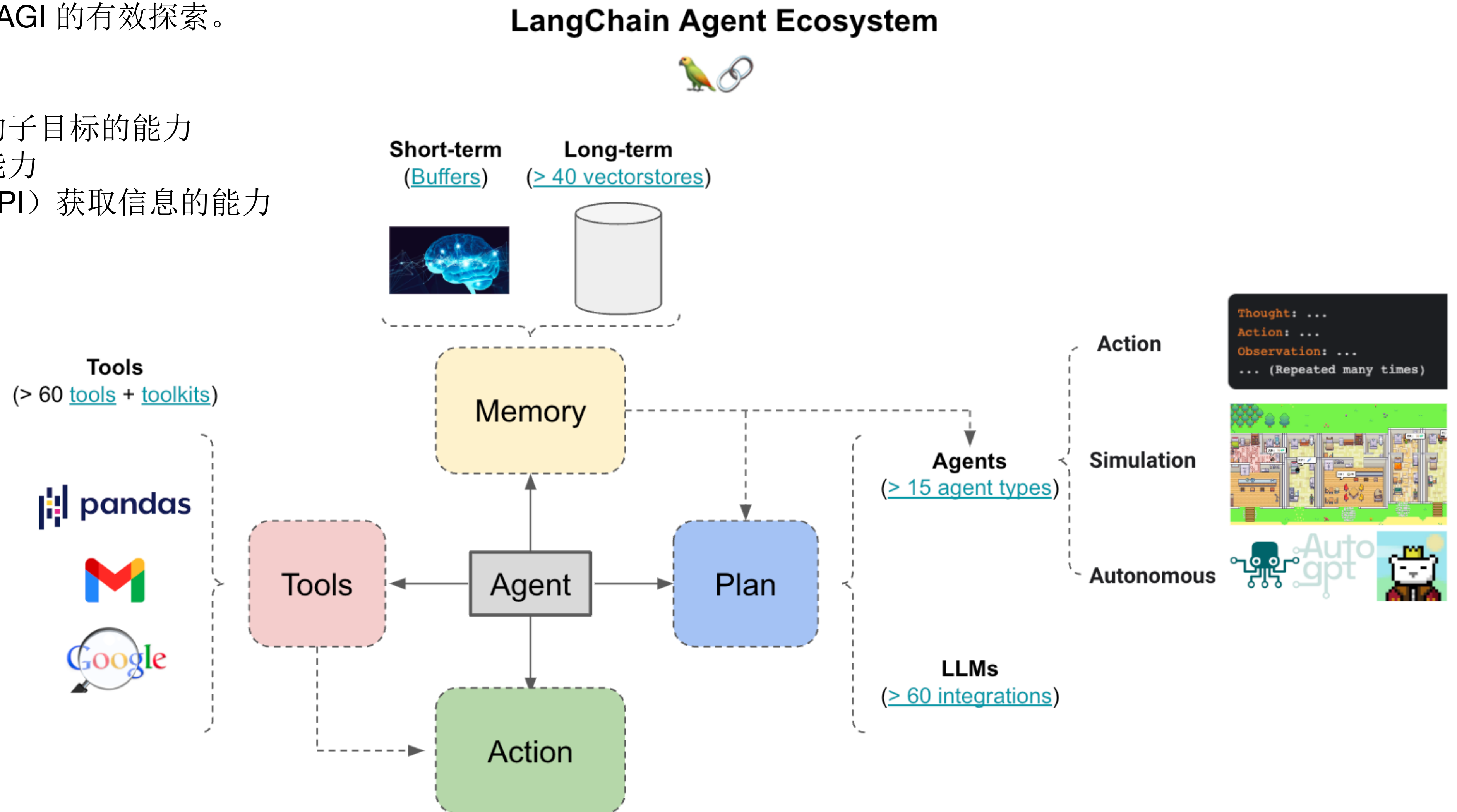
最主要的3个组件：

- 规划：将任务分解为较小的子目标的能力
- 记忆：保留和回忆信息的能力
- 工具：从外部来源（例如API）获取信息的能力

相比 LLM + API 的应用。

智能代理具有3点优势：

- 自我纠正
- 处理多跳任务
- 解决长期任务



深入理解 LangChain 智能代理

LangChain Agent Ecosystem



规划（Planning）

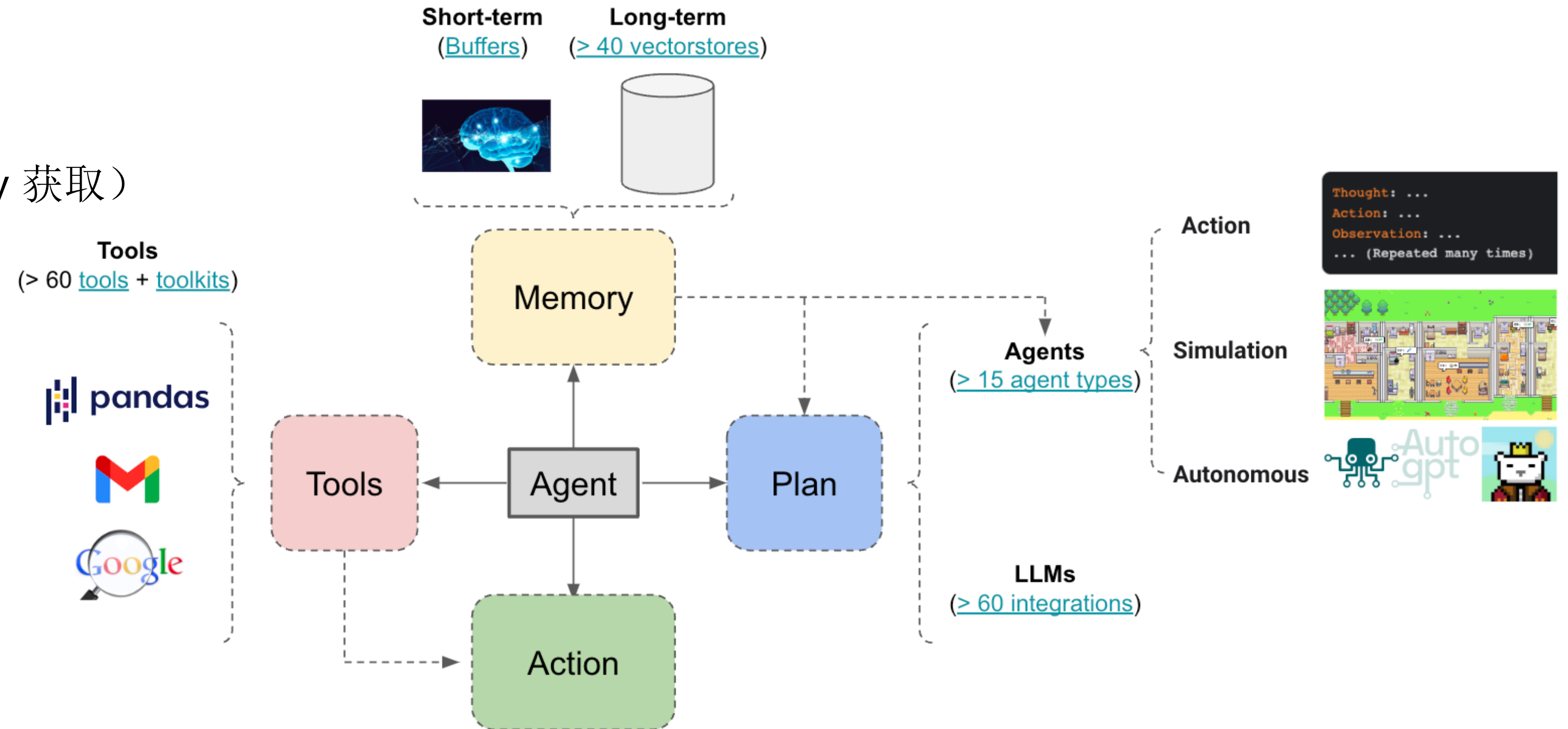
- 提示（Prompt）：
 - LLM 多角色赋能
 - 给予充分的上下文（例：从 Memory 获取）
 - 学习策略（例：思维链 CoT）
- 代理（Agent）：决策下一步做什么

记忆（Memory）

- 短期（Short-term）：内存
- 长期（Long-term）：向量数据库

工具（Tools）

- 百花齐放的外部可调用服务



深入理解 LangChain 智能代理

LangChain Agent Ecosystem



规划（Planning）

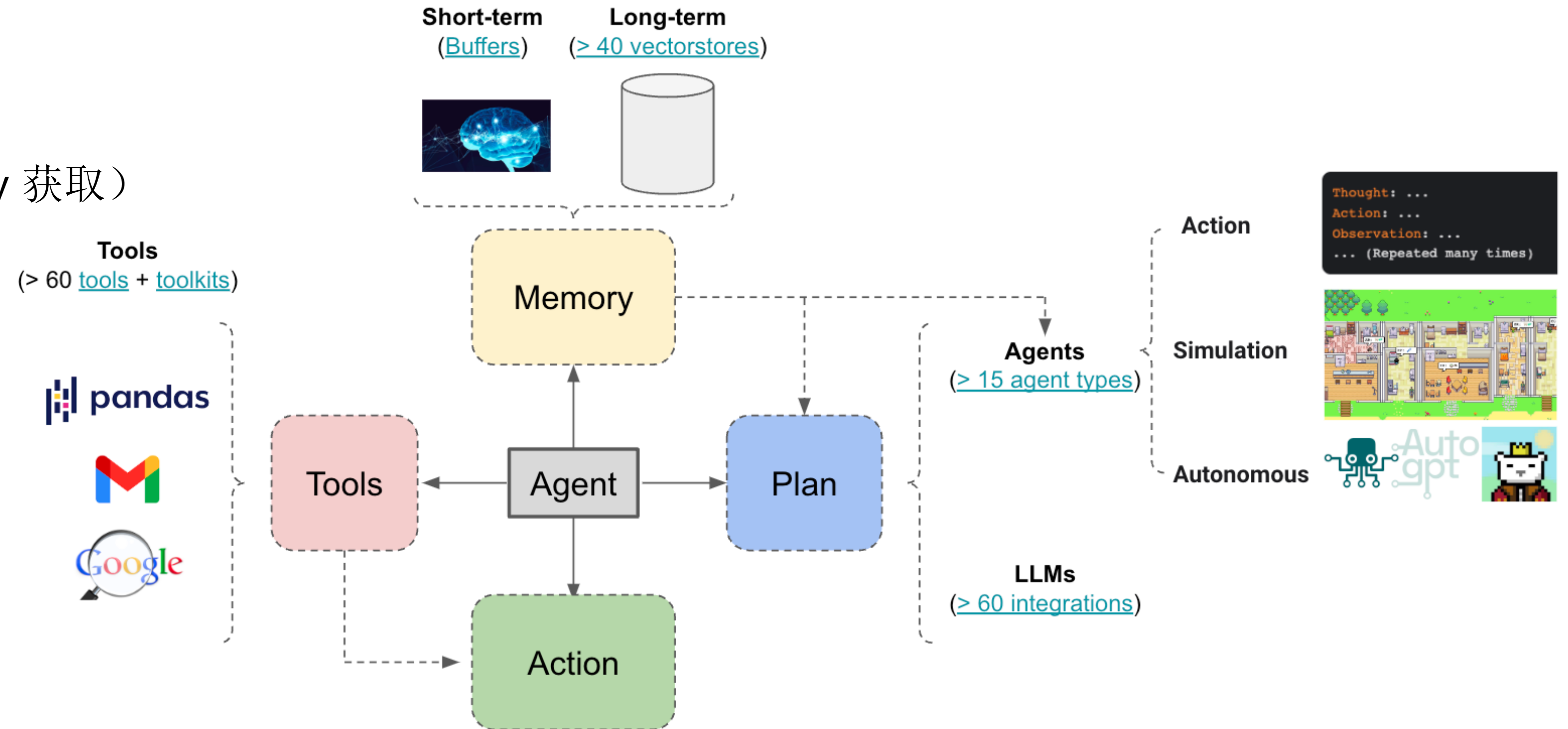
- 提示（Prompt）：
 - LLM 多角色赋能
 - 给予充分的上下文（例：从 Memory 获取）
 - 学习策略（例：思维链 CoT）
- 代理（Agent）：决策下一步做什么

记忆（Memory）

- 短期（Short-term）：内存
- 长期（Long-term）：向量数据库

工具（Tools）

- 百花齐放的外部可调用服务



智能代理分类：

- 行动代理（Action agents）：旨在决定行动序列（工具使用）（例如OpenAI Function Call，ReAct）。
- 模拟代理（Simulation agents）：通常设计用于角色扮演，在模拟环境中进行（例如生成式智能体，CAMEL）。
- 自主智能体（Autonomous agent）：旨在独立执行以实现长期目标（例如Auto-GPT，BabyAGI，）。

master

langchain / libs / experimental / langchain_experimental /

Go to file

t

Add file

nikhilkjha

Initial commit for comprehend moderator (#9665)

✓

d57d08f · yesterday

History

Name	Last commit message	Last commit date
..		
autonomous_agents	Use a submodule for pydantic v1 compat (#9371)	last week
comprehend_moderation	Initial commit for comprehend moderator (#9665)	yesterday
cpal	poetry lock the experimental package. (#9478)	5 days ago
generative_agents	Use a submodule for pydantic v1 compat (#9371)	last week
llms	Use a submodule for pydantic v1 compat (#9371)	last week
pal_chain	Use a submodule for pydantic v1 compat (#9371)	last week
plan_and_execute	Use a submodule for pydantic v1 compat (#9371)	last week
prompts	Harri official pre-release (#9106)	last month
pydantic_v1	poetry lock the experimental package. (#9478)	5 days ago
smart_llm	Use a submodule for pydantic v1 compat (#9371)	last week
sql	Use a submodule for pydantic v1 compat (#9371)	last week

LangChain Experimental 模块

☰

langchain-ai / langchain

🔍 Type / to search

>

+ ▾

🔄

🔗

📁

👤

<> CodeIssues2.3kPull requests336DiscussionsActionsProjectsSecurityInsights

📁

🔗 master ▾

langchain / libs / experimental / langchain_experimental / autonomous_agents / 📄

🔍 Go to file

t

Add file ▾

...

👤 nfcampos

Use a submodule for pydantic v1 compat (#9371) 🗨️ ✓

c0d6742 · last week

🕒 History

Name	Last commit message	Last commit date
📁 ..		
📁 autogpt	Use a submodule for pydantic v1 compat (#9371)	last week
📁 baby_agi	Use a submodule for pydantic v1 compat (#9371)	last week
📁 hugginggpt	Use a submodule for pydantic v1 compat (#9371)	last week
📄 __init__.py	Harrison/hugginggpt (#8162)	last month

自主智能体 (Autonomous agent)

master

langchain / libs / experimental / langchain_experimental / autonomous_agents / autogpt

Go to file

t

Add file

nfcampos

Use a submodule for pydantic v1 compat (#9371)

✓

c0d6742 · last week

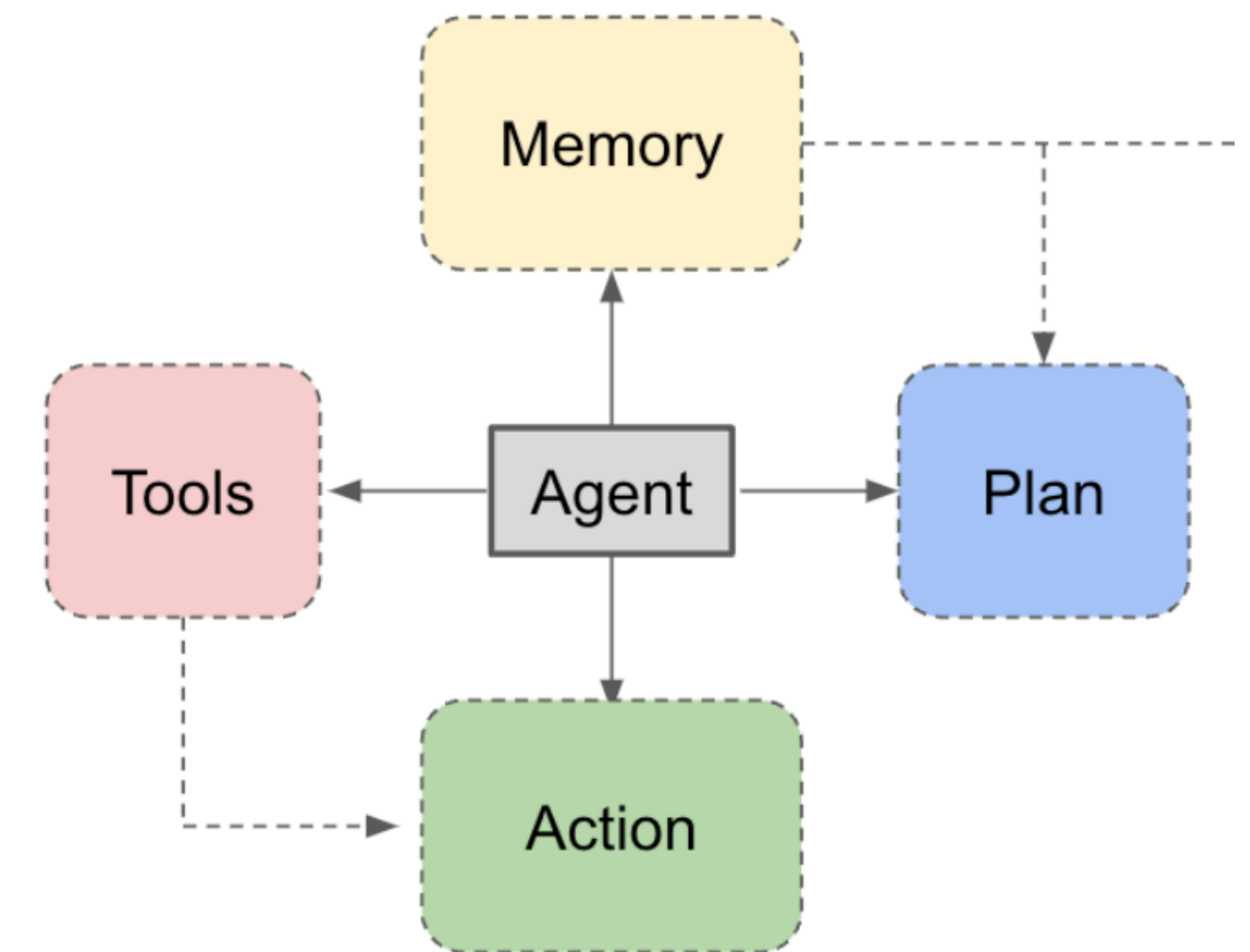
History

Name	Last commit message	Last commit date
..		
__init__.py	Harrison/official pre release (#8106)	last month
agent.py	Use a submodule for pydantic v1 compat (#9371)	last week
memory.py	Use a submodule for pydantic v1 compat (#9371)	last week
output_parser.py	update experimental (#8402)	last month
prompt.py	Use a submodule for pydantic v1 compat (#9371)	last week
prompt_generator.py	update experimental (#8402)	last month

LangChain 版 Auto-GPT

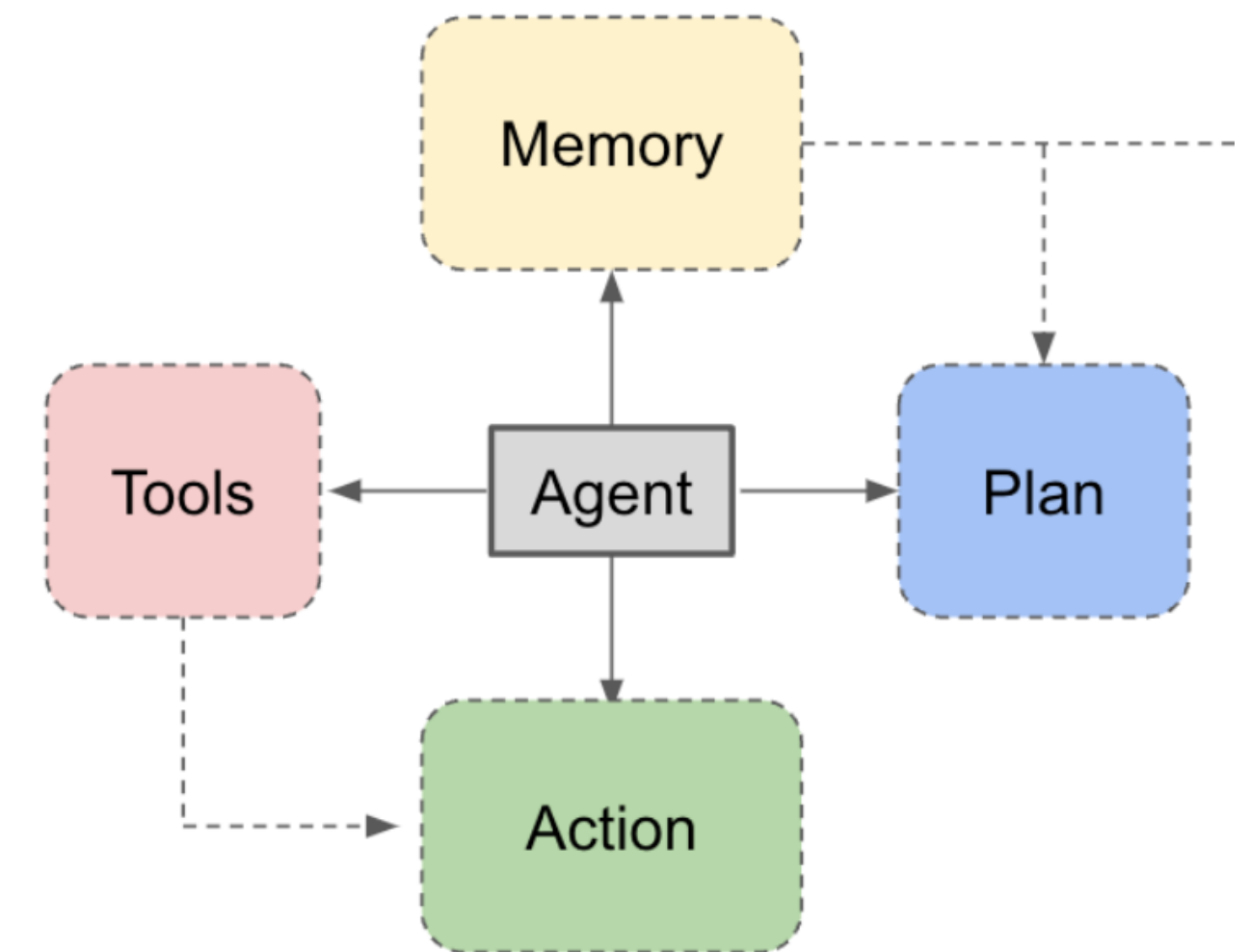
```
class AutoGPT:
    """Agent class for interacting with Auto-GPT."""

    def __init__(
        self,
        ai_name: str,
        memory: VectorStoreRetriever,
        chain: LLMChain,
        output_parser: BaseAutoGPTOutputParser,
        tools: List[BaseTool],
        feedback_tool: Optional[HumanInputRun] = None,
        chat_history_memory: Optional[BaseChatMessageHistory] = None,
    ):
        self.ai_name = ai_name
        self.memory = memory
        self.next_action_count = 0
        self.chain = chain
        self.output_parser = output_parser
        self.tools = tools
        self.feedback_tool = feedback_tool
        self.chat_history_memory = chat_history_memory or ChatMessageHistory()
```



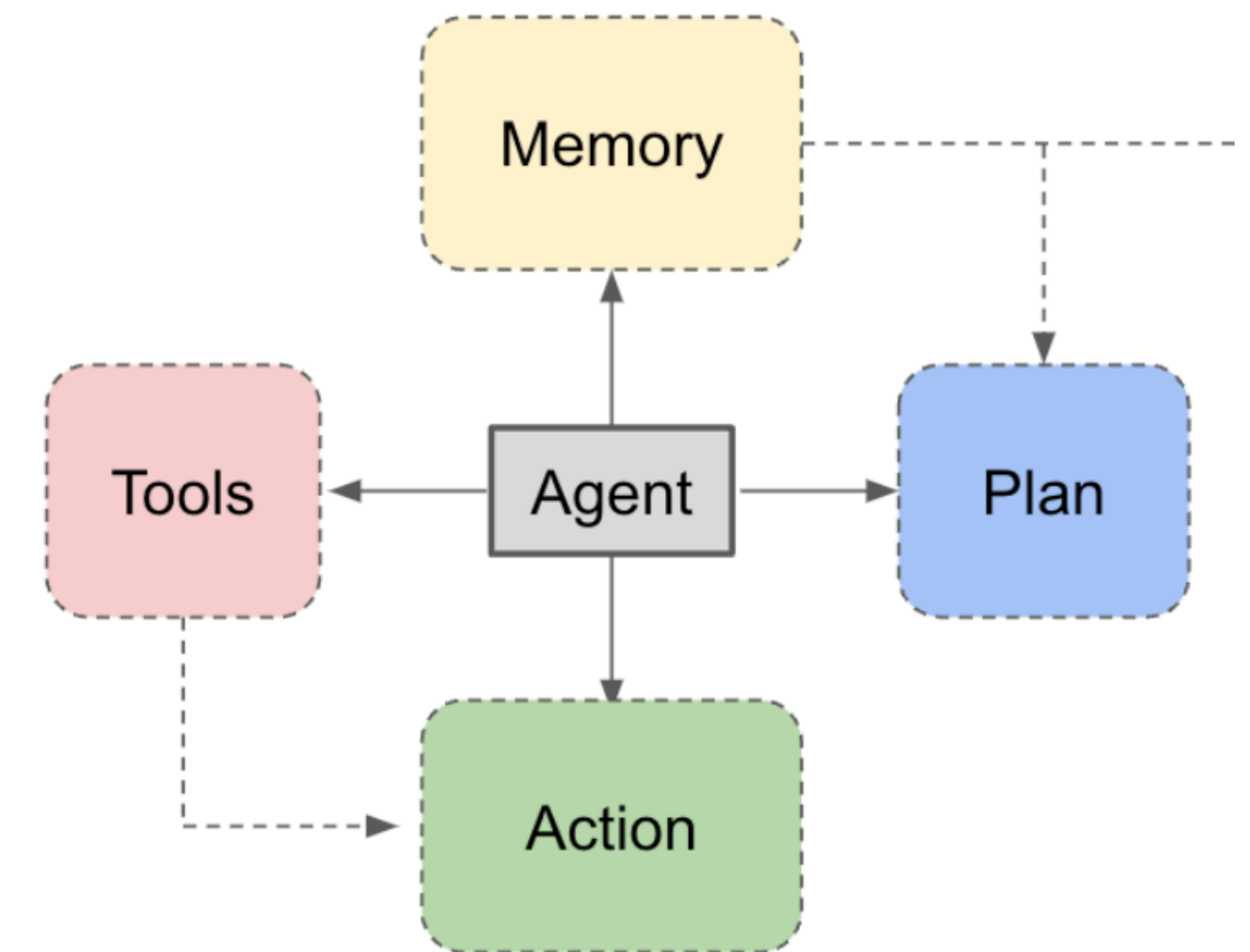
Auto-GPT 自主智能体设计：经典构造 from_llm_and_tools

```
@classmethod
def from_llm_and_tools(
    cls,
    ai_name: str,
    ai_role: str,
    memory: VectorStoreRetriever,
    tools: List[BaseTool],
    llm: BaseChatModel,
    human_in_the_loop: bool = False,
    output_parser: Optional[BaseAutoGPTOutputParser] = None,
    chat_history_memory: Optional[BaseChatMessageHistory] = None,
) -> AutoGPT:
    prompt = AutoGPTPrompt(
        ai_name=ai_name,
        ai_role=ai_role,
        tools=tools,
        input_variables=["memory", "messages", "goals", "user_input"],
        token_counter=llm.get_num_tokens,
    )
    human_feedback_tool = HumanInputRun() if human_in_the_loop else None
    chain = LLMChain(llm=llm, prompt=prompt)
    return cls(
        ai_name,
        memory,
        chain,
        output_parser or AutoGPTOutputParser(),
        tools,
        feedback_tool=human_feedback_tool,
        chat_history_memory=chat_history_memory,
    )
```



Auto-GPT 自主智能体设计：经典构造 from_llm_and_tools

```
@classmethod
def from_llm_and_tools(
    cls,
    ai_name: str,
    ai_role: str,
    memory: VectorStoreRetriever,
    tools: List[BaseTool],
    llm: BaseChatModel,
    human_in_the_loop: bool = False,
    output_parser: Optional[BaseAutoGPTOutputParser] = None,
    chat_history_memory: Optional[BaseChatMessageHistory] = None,
) -> AutoGPT:
    prompt = AutoGPTPrompt(
        ai_name=ai_name,
        ai_role=ai_role,
        tools=tools,
        input_variables=["memory", "messages", "goals", "user_input"],
        token_counter=llm.get_num_tokens,
    )
    human_feedback_tool = HumanInputRun() if human_in_the_loop else None
    chain = LLMChain(llm=llm, prompt=prompt)
    return cls(
        ai_name,
        memory,
        chain,
        output_parser or AutoGPTOutputParser(),
        tools,
        feedback_tool=human_feedback_tool,
        chat_history_memory=chat_history_memory,
    )
```



Auto-GPT 自主智能体设计：核心方法 run

```
def run(self, goals: List[str]) -> str:
    user_input = (
        "Determine which next command to use, "
        "and respond using the format specified above:"
    )
    # Interaction Loop
    loop_count = 0
    while True:
        # Discontinue if continuous limit is reached
        loop_count += 1

        # Send message to AI, get response
        assistant_reply = self.chain.run(
            goals=goals,
            messages=self.chat_history_memory.messages,
            memory=self.memory,
            user_input=user_input,
        )

        # Print Assistant thoughts
        print(assistant_reply)
        self.chat_history_memory.add_message(HumanMessage(content=user_input))
        self.chat_history_memory.add_message(AIMessage(content=assistant_reply))

        # Get command name and arguments
        action = self.output_parser.parse(assistant_reply)
        tools = {t.name: t for t in self.tools}
        if action.name == FINISH_NAME:
            return action.args["response"]
        if action.name in tools:
            tool = tools[action.name]
            try:
                observation = tool.run(action.args)
```

```
            if action.name in tools:
                tool = tools[action.name]
                try:
                    observation = tool.run(action.args)
                except ValidationError as e:
                    observation = (
                        f"Validation Error in args: {str(e)}, args: {action.args}"
                    )
                except Exception as e:
                    observation = (
                        f"Error: {str(e)}, {type(e).__name__}, args: {action.args}"
                    )
                result = f"Command {tool.name} returned: {observation}"
            elif action.name == "ERROR":
                result = f"Error: {action.args}. "
            else:
                result = (
                    f"Unknown command '{action.name}'. "
                    f"Please refer to the 'COMMANDS' list for available "
                    f"commands and only respond in the specified JSON format."
                )

            memory_to_add = (
                f"Assistant Reply: {assistant_reply} " f"\nResult: {result} "
            )
            if self.feedback_tool is not None:
                feedback = f"\n{self.feedback_tool.run('Input: ')}"
                if feedback in {"q", "stop"}:
                    print("EXITING")
                    return "EXITING"
                memory_to_add += feedback

            self.memory.add_documents([Document(page_content=memory_to_add)])
            self.chat_history_memory.add_message(SystemMessage(content=result))
```


Auto-GPT 自主智能体设计：核心方法 run

```
def run(self, goals: List[str]) -> str:
    user_input = (
        "Determine which next command to use, "
        "and respond using the format specified above:"
    )

    # Interaction Loop
    loop_count = 0
    while True:
        # Discontinue if continuous limit is reached
        loop_count += 1

        # Send message to AI, get response
        assistant_reply = self.chain.run(
            goals=goals,
            messages=self.chat_history_memory.messages,
            memory=self.memory,
            user_input=user_input,
        )

        # Print Assistant thoughts
        print(assistant_reply)
        self.chat_history_memory.add_message(HumanMessage(content=user_input))
        self.chat_history_memory.add_message(AIMessage(content=assistant_reply))

        # Get command name and arguments
        action = self.output_parser.parse(assistant_reply)
        tools = {t.name: t for t in self.tools}

        if action.name == FINISH_NAME:
            return action.args["response"]
        if action.name in tools:
            tool = tools[action.name]

            try:
                observation = tool.run(action.args)
```

```
                if action.name in tools:
                    tool = tools[action.name]
                    try:
                        observation = tool.run(action.args)
                    except ValidationError as e:
                        observation = (
                            f"Validation Error in args: {str(e)}, args: {action.args}"
                        )
                    except Exception as e:
                        observation = (
                            f"Error: {str(e)}, {type(e).__name__}, args: {action.args}"
                        )
                    result = f"Command {tool.name} returned: {observation}"
                elif action.name == "ERROR":
                    result = f"Error: {action.args}. "
                else:
                    result = (
                        f"Unknown command '{action.name}'. "
                        f"Please refer to the 'COMMANDS' list for available "
                        f"commands and only respond in the specified JSON format."
                    )

                memory_to_add = (
                    f"Assistant Reply: {assistant_reply} " f"\nResult: {result} "
                )

                if self.feedback_tool is not None:
                    feedback = f"\n{self.feedback_tool.run('Input: ')}"
                    if feedback in {"q", "stop"}:
                        print("EXITING")
                        return "EXITING"
                    memory_to_add += feedback

                self.memory.add_documents([Document(page_content=memory_to_add)])
                self.chat_history_memory.add_message(SystemMessage(content=result))
```


Auto-GPT Prompt 设计：提示词元模板

```
from langchain_experimental.autonomous_agents.autogpt.prompt_generator import get_prompt
from langchain_experimental.pydantic_v1 import BaseModel

class AutoGPTPrompt(BaseChatPromptTemplate, BaseModel):
    """Prompt for AutoGPT."""

    ai_name: str
    ai_role: str
    tools: List[BaseTool]
    token_counter: Callable[[str], int]
    send_token_limit: int = 4196

    def construct_full_prompt(self, goals: List[str]) -> str:
        prompt_start = (
            "Your decisions must always be made independently "
            "without seeking user assistance.\n"
            "Play to your strengths as an LLM and pursue simple "
            "strategies with no legal complications.\n"
            "If you have completed all your tasks, make sure to "
            "'use the \"finish\" command.'
        )
        # Construct full prompt
        full_prompt = (
            f"You are {self.ai_name}, {self.ai_role}\n{prompt_start}\n\nGOALS:\n\n"
        )
        for i, goal in enumerate(goals):
            full_prompt += f"{i+1}. {goal}\n"

        full_prompt += f"\n\n{get_prompt(self.tools)}"
        return full_prompt
```

```
def format_messages(self, **kwargs: Any) -> List[BaseMessage]:
    base_prompt = SystemMessage(content=self.construct_full_prompt(kwargs["goals"]))
    time_prompt = SystemMessage(
        content=f"The current time and date is {time.strftime('%c')}}"
    )
    used_tokens = self.token_counter(base_prompt.content) + self.token_counter(
        time_prompt.content
    )
    memory: VectorStoreRetriever = kwargs["memory"]
    previous_messages = kwargs["messages"]
    relevant_docs = memory.get_relevant_documents(str(previous_messages[-10:]))
    relevant_memory = [d.page_content for d in relevant_docs]
    relevant_memory_tokens = sum(
        [self.token_counter(doc) for doc in relevant_memory]
    )
    while used_tokens + relevant_memory_tokens > 2500:
        relevant_memory = relevant_memory[:-1]
        relevant_memory_tokens = sum(
            [self.token_counter(doc) for doc in relevant_memory]
        )
    content_format = (
        f"This reminds you of these events "
        f"from your past:\n{relevant_memory}\n\n"
    )
    memory_message = SystemMessage(content=content_format)
    used_tokens += self.token_counter(memory_message.content)
    historical_messages: List[BaseMessage] = []
    for message in previous_messages[-10:][::-1]:
        message_tokens = self.token_counter(message.content)
        if used_tokens + message_tokens > self.send_token_limit - 1000:
            break
        historical_messages = [message] + historical_messages
        used_tokens += message_tokens
    input_message = HumanMessage(content=kwargs["user_input"])
    messages: List[BaseMessage] = [base_prompt, time_prompt, memory_message]
    messages += historical_messages
    messages.append(input_message)
    return messages
```


Auto-GPT Prompt 设计: token 与 messages

```
from langchain_experimental.autonomous_agents.autogpt.prompt_generator import get_prompt
from langchain_experimental.pydantic_v1 import BaseModel

class AutoGPTPrompt(BaseChatPromptTemplate, BaseModel):
    """Prompt for AutoGPT."""

    ai_name: str
    ai_role: str
    tools: List[BaseTool]
    token_counter: Callable[[str], int]
    send_token_limit: int = 4196

    def construct_full_prompt(self, goals: List[str]) -> str:
        prompt_start = (
            "Your decisions must always be made independently "
            "without seeking user assistance.\n"
            "Play to your strengths as an LLM and pursue simple "
            "strategies with no legal complications.\n"
            "If you have completed all your tasks, make sure to "
            'use the "finish" command.'
        )
        # Construct full prompt
        full_prompt = (
            f"You are {self.ai_name}, {self.ai_role}\n{prompt_start}\n\nGOALS:\n\n"
        )
        for i, goal in enumerate(goals):
            full_prompt += f"{i+1}. {goal}\n"

        full_prompt += f"\n\n{get_prompt(self.tools)}"
        return full_prompt
```

```
def format_messages(self, **kwargs: Any) -> List[BaseMessage]:
    base_prompt = SystemMessage(content=self.construct_full_prompt(kwargs["goals"]))
    time_prompt = SystemMessage(
        content=f"The current time and date is {time.strftime('%c')}}"
    )
    used_tokens = self.token_counter(base_prompt.content) + self.token_counter(
        time_prompt.content
    )

    memory: VectorStoreRetriever = kwargs["memory"]
    previous_messages = kwargs["messages"]
    relevant_docs = memory.get_relevant_documents(str(previous_messages[-10:]))
    relevant_memory = [d.page_content for d in relevant_docs]
    relevant_memory_tokens = sum(
        [self.token_counter(doc) for doc in relevant_memory]
    )
    while used_tokens + relevant_memory_tokens > 2500:
        relevant_memory = relevant_memory[:-1]
        relevant_memory_tokens = sum(
            [self.token_counter(doc) for doc in relevant_memory]
        )
    content_format = (
        f"This reminds you of these events "
        f"from your past:\n{relevant_memory}\n\n"
    )
    memory_message = SystemMessage(content=content_format)
    used_tokens += self.token_counter(memory_message.content)

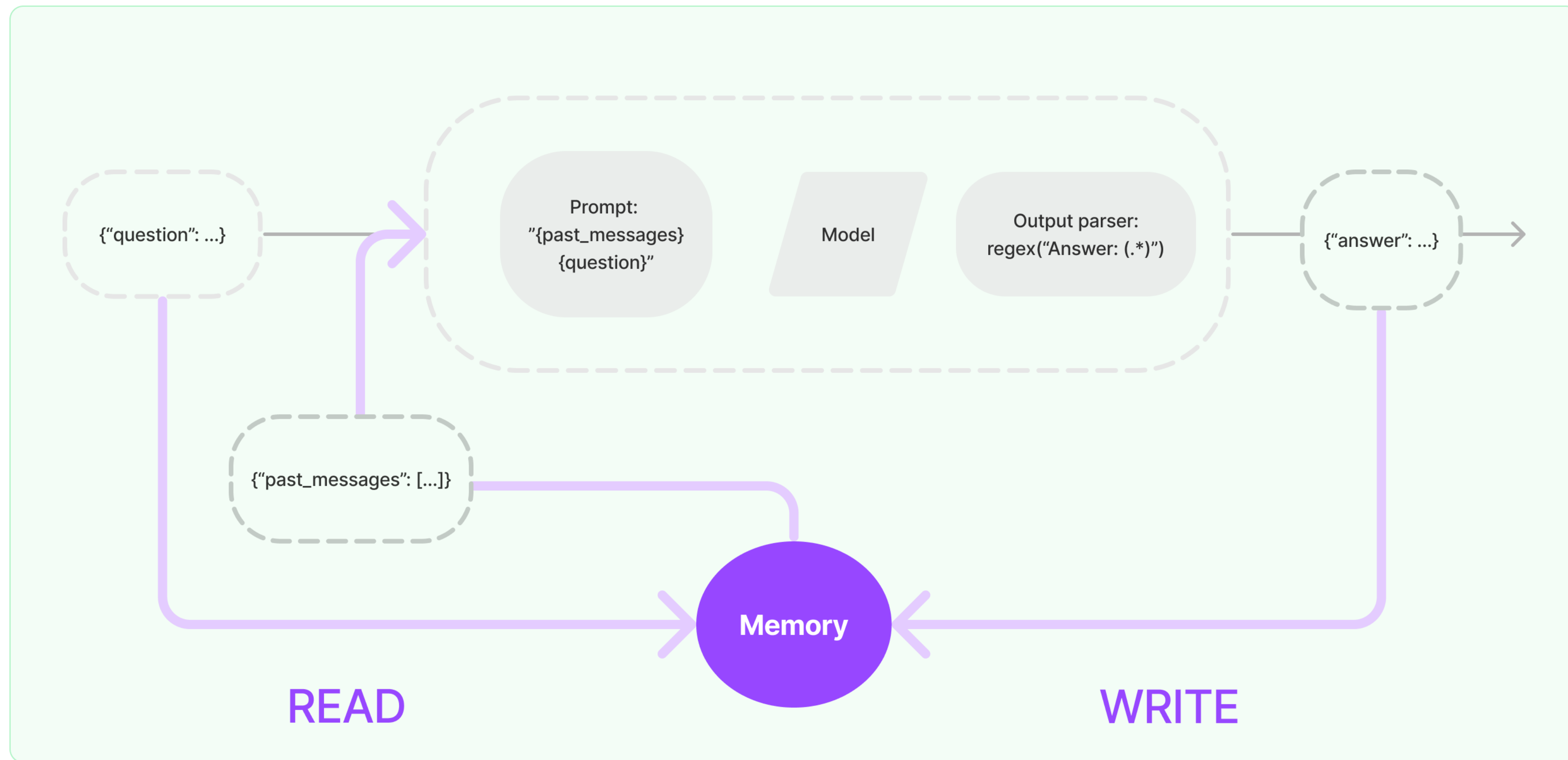
    historical_messages: List[BaseMessage] = []
    for message in previous_messages[-10:][::-1]:
        message_tokens = self.token_counter(message.content)
        if used_tokens + message_tokens > self.send_token_limit - 1000:
            break
        historical_messages = [message] + historical_messages
        used_tokens += message_tokens
    input_message = HumanMessage(content=kwargs["user_input"])
    messages: List[BaseMessage] = [base_prompt, time_prompt, memory_message]
    messages += historical_messages
    messages.append(input_message)
    return messages
```

使用向量数据库实现长期记忆

Memory 总量控制

Messages 构造完成

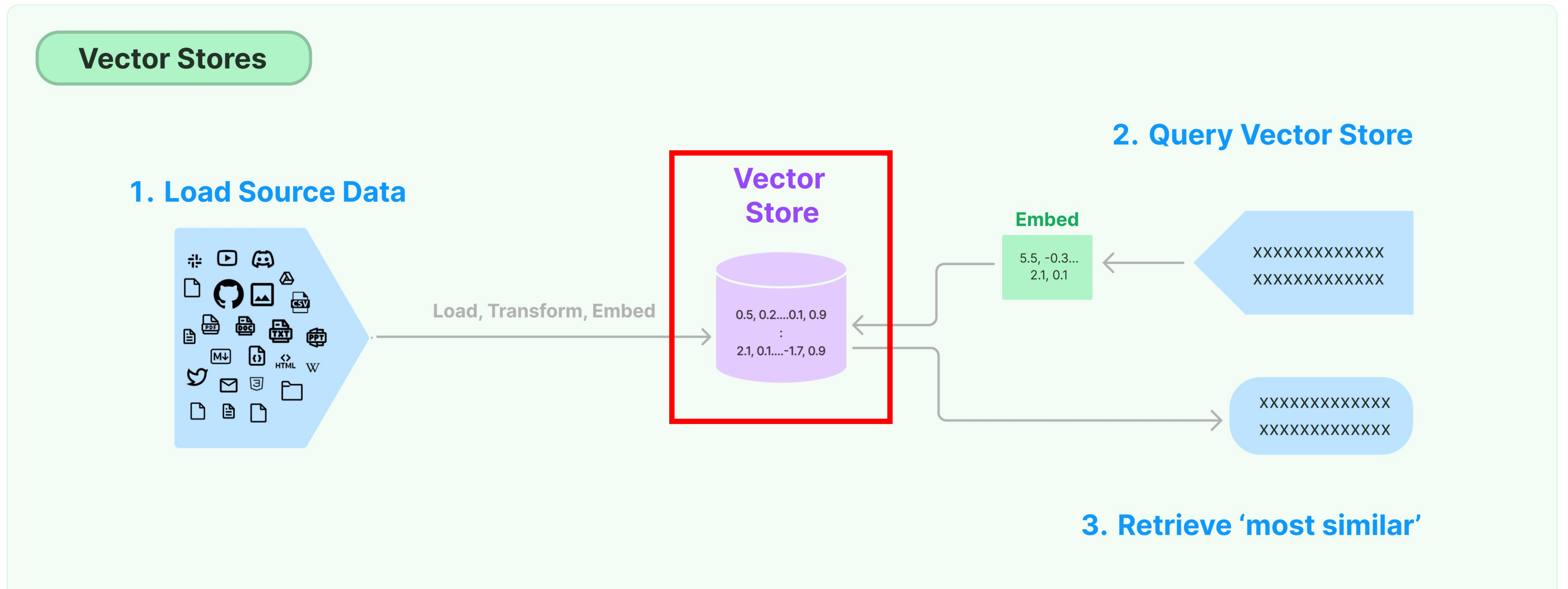
温故：LangChain Memory 模块



Auto-GPT Memory 设计

```
class VectorStoreRetriever(BaseRetriever):  
    def _get_relevant_documents(  
        self, query: str, *, run_manager: CallbackManagerForRetrieverRun  
    ) -> List[Document]:  
        if self.search_type == "similarity":  
            docs = self.vectorstore.similarity_search(query, **self.search_kwargs)  
        elif self.search_type == "similarity_score_threshold": 相似度搜索  
            docs_and_similarities = (  
                self.vectorstore.similarity_search_with_relevance_scores(  
                    query, **self.search_kwargs 带阈值的相似度搜索  
                )  
            )  
            docs = [doc for doc, _ in docs_and_similarities]  
        elif self.search_type == "mmr":  
            docs = self.vectorstore.max_marginal_relevance_search(  
                query, **self.search_kwargs 最大边际相关性  
            )  
        else:  
            raise ValueError(f"search_type of {self.search_type} not allowed.")  
        return docs
```

```
def format_messages(self, **kwargs: Any) -> List[BaseMessage]:  
    base_prompt = SystemMessage(content=self.construct_full_prompt(kwargs["goals"]))  
    time_prompt = SystemMessage(  
        content=f"The current time and date is {time.strftime('%c')}}"  
    )  
    used_tokens = self.token_counter(base_prompt.content) + self.token_counter(  
        time_prompt.content  
    )  
    memory: VectorStoreRetriever = kwargs["memory"]  
    previous_messages = kwargs["messages"]  
    relevant_docs = memory.get_relevant_documents(str(previous_messages[-10:]))  
    relevant_memory = [d.page_content for d in relevant_docs]  
    relevant_memory_tokens = sum(  
        [self.token_counter(doc) for doc in relevant_memory]  
    )  
    while used_tokens + relevant_memory_tokens > 2500:  
        relevant_memory = relevant_memory[:-1]  
        relevant_memory_tokens = sum(  
            [self.token_counter(doc) for doc in relevant_memory]  
        )  
    content_format = (  
        f"This reminds you of these events "  
        f"from your past:\n{relevant_memory}\n\n"  
    )  
    memory_message = SystemMessage(content=content_format)  
    used_tokens += self.token_counter(memory_message.content)  
    historical_messages: List[BaseMessage] = []  
    for message in previous_messages[-10:][::-1]:  
        message_tokens = self.token_counter(message.content)  
        if used_tokens + message_tokens > self.send_token_limit - 1000:  
            break  
        historical_messages = [message] + historical_messages  
        used_tokens += message_tokens  
    input_message = HumanMessage(content=kwargs["user_input"])  
    messages: List[BaseMessage] = [base_prompt, time_prompt, memory_message]  
    messages += historical_messages  
    messages.append(input_message)  
    return messages
```

Embed

0.5, 0.2....0.1, 0.9
-0.1, 0.4....1.4, 5.9

0.2, 0.7....2.1, -1.2
4.1, 3.4....-1.5, 2.5

5.5, -0.3....0.8, 2.3
2.1, 0.1....-1.7, 0.9

Store

0.5, 0.2....0.1, 0.9
:
2.1, 0.1....-1.7, 0.9

Vector Stores: 向量数据库

- Alibaba Cloud OpenSearch
- AnalyticDB
- Annoy
- Atlas
- AwaDB
- Azure Cognitive Search
- BagelDB
- Cassandra
- Chroma
- Clarifai
- ClickHouse Vector Search
- Activeloop's Deep Lake
- Dingo
- DocArrayHnswSearch
- DocArrayInMemorySearch
- ElasticSearch
- **FAISS**
- Hologres
- LanceDB
- Marqo
- MatchingEngine
- Meilisearch
- Milvus
- MongoDB Atlas
- MyScale
- OpenSearch
- pg_embedding
- PGVector
- Pinecone
- Qdrant
- Redis
- Rockset
- ScaNN
- SingleStoreDB
- scikit-learn
- StarRocks
- Supabase (Postgres)
- Tair
- Tigris
- Typesense
- USearch
- Vectara
- Weaviate
- Xata
- Zilliz

Faiss是一个用于高效相似性搜索和密集向量聚类的库，由 **Meta** 公司的基础人工智能研究小组开发。

- 它的算法支持在任意大小的向量集合中进行搜索，甚至可以处理无法放入内存的向量集合。
- 它还包含了用于评估和参数调整的支持代码。
- **Faiss**使用C++编写，并提供完整的Python/numpy封装，其中一些最常用的算法已经在GPU上实现。
- 并且，**Faiss** 提供了 CPU 和 GPU 两种版本的安装包。

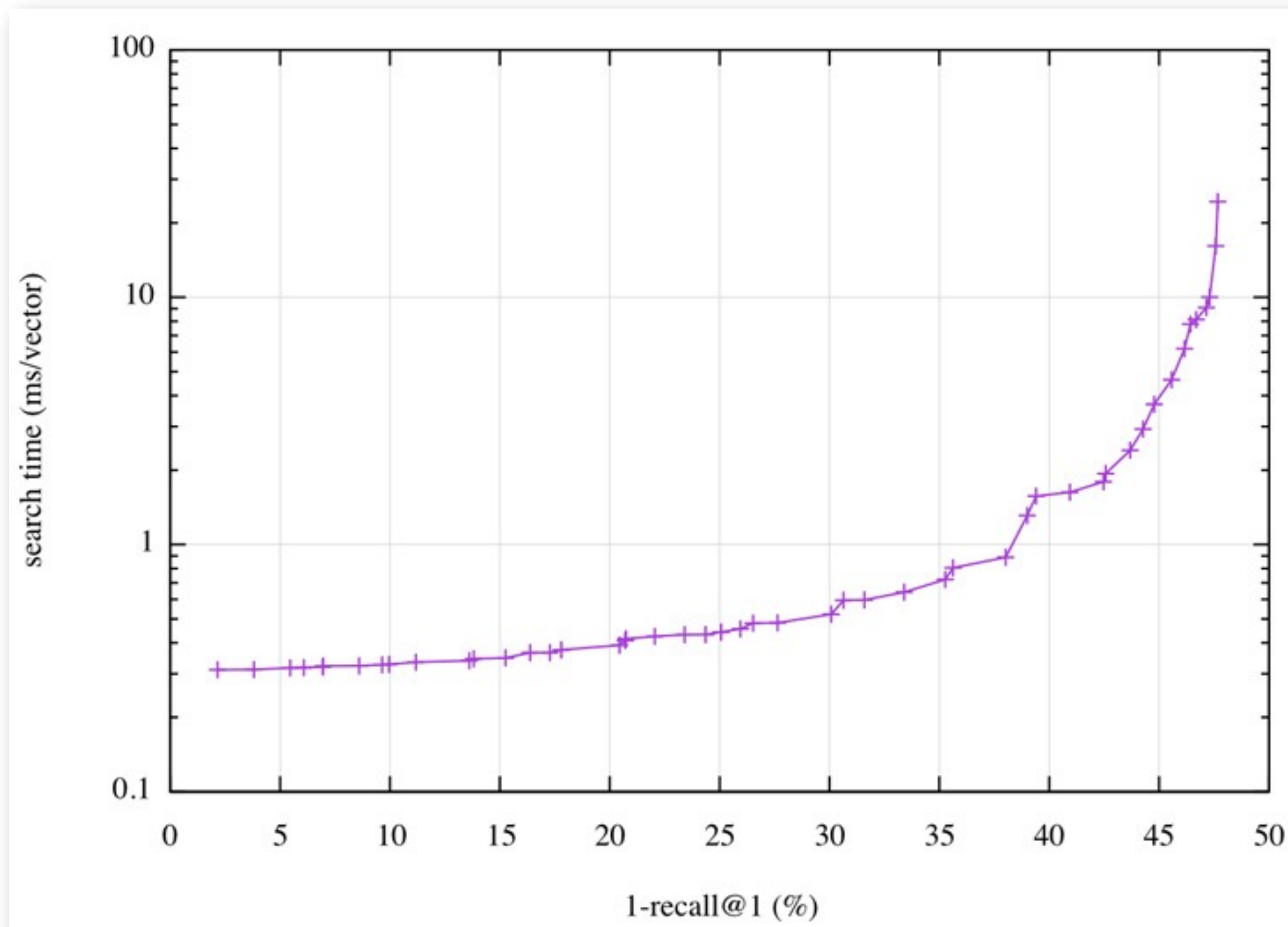
Faiss GPU 版本：

- 支持CPU 或 GPU 的输入。
- GPU索引（**indexes**）可以即插即用替代CPU索引（例如，用GpuIndexFlatL2替换IndexFlatL2），并且对于GPU内存的复制操作会自动处理。
- 如果输入和输出都保留在GPU上，则查询性能更好。
- 支持单个和多个GPU的混合使用。

Faiss 性能优异:

- 在每个向量的查询时间小于2毫秒的情况下，获得40%的**1-recall@1**;
- 在0.5毫秒的时间预算内达到30%。
- 2毫秒的查询时间相当于单核心每秒500次查询 (QPS)。

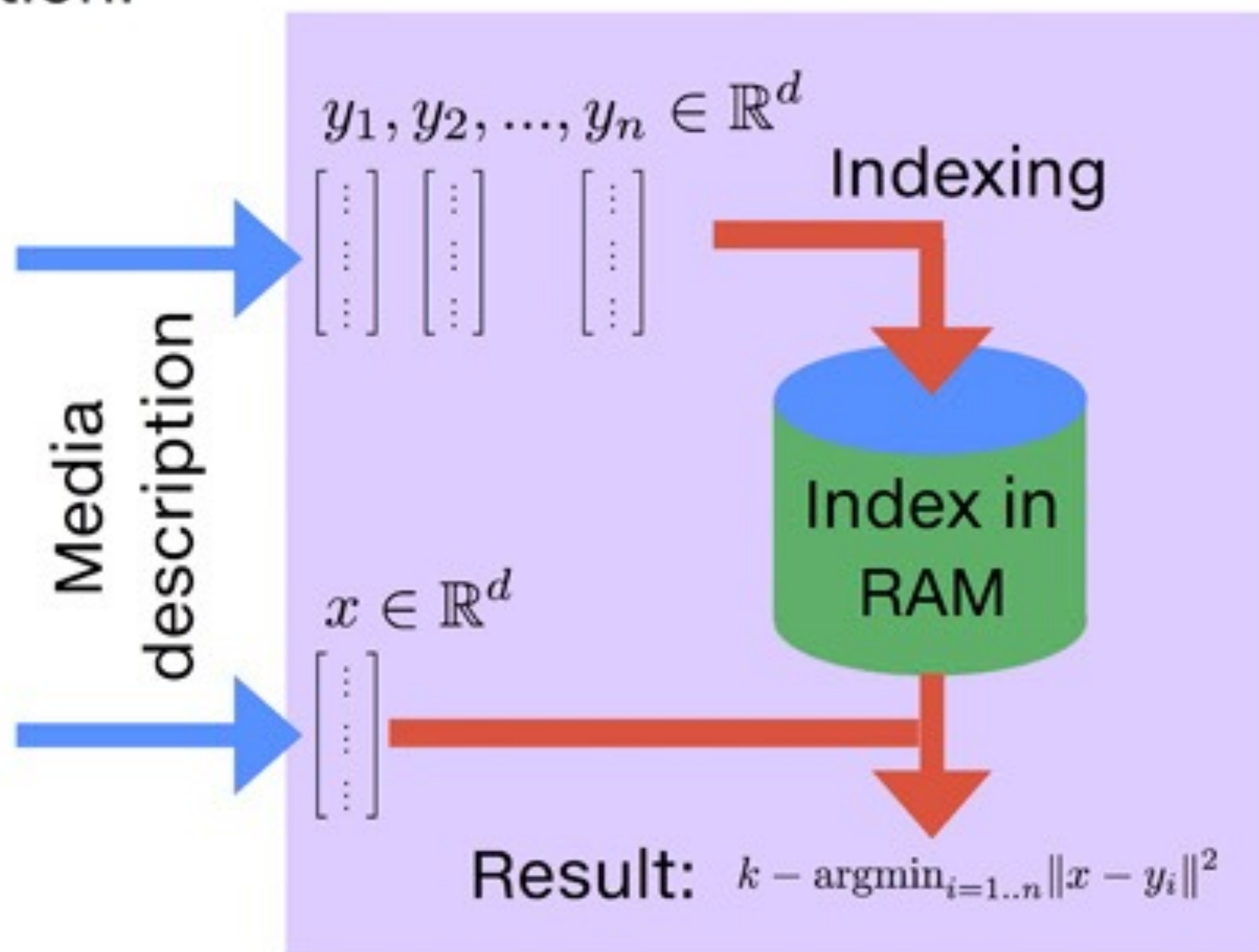
Faiss 性能与该领域最先进研究成果进行比较:
“Efficient Indexing of Billion-Scale Datasets of Deep Descriptors”由Babenko和Lempitsky在CVPR 2016中发表，他们需要20毫秒才能获得45%的1-recall@1。



Build index for a collection:



Query:



Auto-GPT OutputParser 设计

```
class AutoGPTOutputParser(BaseAutoGPTOutputParser):
    """Output parser for AutoGPT."""

    def parse(self, text: str) -> AutoGPTAction:
        try:
            parsed = json.loads(text, strict=False)
        except json.JSONDecodeError:
            preprocessed_text = preprocess_json_input(text)
            try:
                parsed = json.loads(preprocessed_text, strict=False)
            except Exception:
                return AutoGPTAction(
                    name="ERROR",
                    args={"error": f"Could not parse invalid json: {text}"},
                )
        try:
            return AutoGPTAction(
                name=parsed["command"]["name"],
                args=parsed["command"]["args"],
            )
        except (KeyError, TypeError):
            # If the command is null or incomplete, return an erroneous tool
            return AutoGPTAction(
                name="ERROR", args={"error": f"Incomplete command args: {parsed}"}
            )
```

```
class AutoGPTAction(NamedTuple):
    """Action returned by AutoGPTOutputParser."""

    name: str
    args: Dict
```

```
class BaseAutoGPTOutputParser(BaseOutputParser):
    """Base Output parser for AutoGPT."""

    @abstractmethod
    def parse(self, text: str) -> AutoGPTAction:
        """Return AutoGPTAction"""
```

```
def preprocess_json_input(input_str: str) -> str:
    """Preprocesses a string to be parsed as json.
```

Replace single backslashes with double backslashes,
while leaving already escaped ones intact.

Args:

input_str: String to be preprocessed

Returns:

Preprocessed string

"""

```
corrected_str = re.sub(
    r'(?<!\\\)\\"(?![\\"\/bfnrt]|u[0-9a-fA-F]{4})', r'\\\\', input_str
)
return corrected_str
```

实战 **LangChain** 版 **Auto-GPT**

DEMO

课程项目：GitHub openai-quickstart

openai-quickstartPublic

UnpinUnwatch 11Fork 171Starred 351

main1 branch0 tagsGo to fileAdd fileCode

DjangoPeng add langchain-refactored version of openai-translator1f66fba 17 hours ago67 commits

chatgpt-plugins	add initial version of chatgpt plugins	3 weeks ago
docs	make layout more readable	last month
langchain	add langchain-refactored version of openai-translator	17 hours ago
openai-translator	Merge branch 'main' of github.com:DjangoPeng/openai-quickstart in...	3 weeks ago
openai_api	Remove fine_food_reviews_with_embeddings_1k_demo.csv	last month
.gitignore	add initial version of agents and data	last week
LICENSE	Initial commit	last month
README-CN.md	docs: add 8/13 schedule to readme (#49)	4 days ago
README.md	docs: add 8/13 schedule to readme (#49)	4 days ago

README.md

OpenAI Quickstart

English | 中文

This project is designed as a one-stop learning resource for anyone interested in large language models and their application in Artificial Intelligence Governance and Control (AIGC) scenarios. By providing theoretical foundations, development basics, and hands-on examples, this project offers comprehensive guidance on these cutting-edge topics.

About

A comprehensive guide to understanding and implementing large language models with hands-on examples using LangChain for AIGC applications.

ReadmeApache-2.0 licenseActivity351 stars11 watching171 forks

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Contributors 8